

# Manual de Despliegue - Proyecto Integrador (UD5)

**Módulo:** Despliegue de Aplicaciones Web **Autor:** Pablo Jiménez Pinto

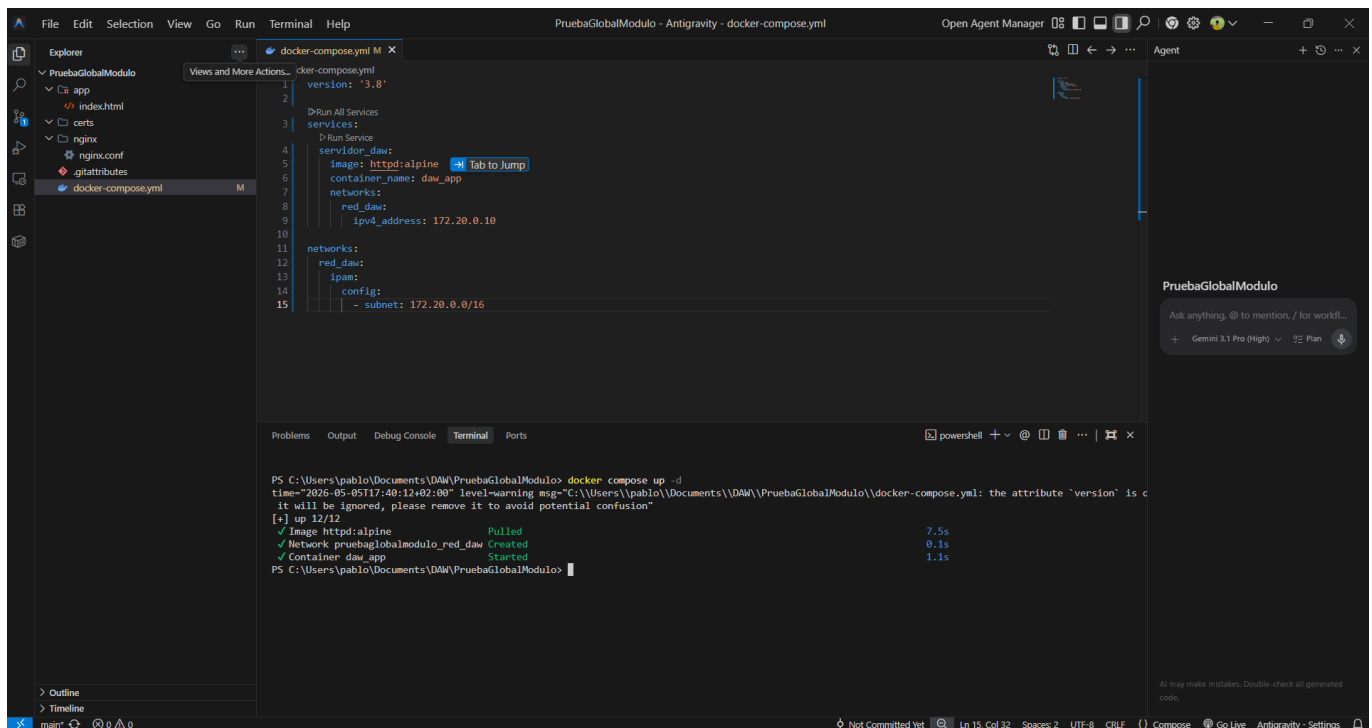
## Apartado 1 (RA1): Arquitectura y Preparación del Entorno

**Objetivo:** Configurar la red del entorno de trabajo con IP estática y levantar el sistema operativo base.

Para realizar este despliegue he optado por una arquitectura basada en contenedores utilizando **Docker**. Esta decisión me permite un aislamiento completo de los servicios y una alta reproducibilidad del entorno.

- **Configuración de la red:** En el archivo `docker-compose.yml` he definido una red personalizada de tipo `bridge` llamada `red_daw`. Le he asignado estáticamente la subred `172.20.0.0/16`. Al contenedor principal (`daw_app`) le he fijado la IP estática `172.20.0.10`.
- **Sistema Operativo Base:** He utilizado la imagen `httpd:alpine`, la cual proporciona un sistema operativo Linux minimalista (Alpine) con el servidor web Apache preinstalado, optimizando así el consumo de recursos.

### Prueba de configuración de la red:



The screenshot shows a Visual Studio Code editor with a file named `docker-compose.yml` open. The file content is as follows:

```
1 version: '3.8'
2
3 services:
4   servidor_daw:
5     image: httpd:alpine
6     container_name: daw_app
7     networks:
8       red_daw:
9         ipv4_address: 172.20.0.10
10
11 networks:
12   red_daw:
13     ipam:
14       config:
15         - subnet: 172.20.0.0/16
```

Below the editor, a terminal window shows the output of the command `docker compose up -d`:

```
PS C:\Users\pablo\Documents\DAW\PruebaGlobalModulo> docker compose up -d
time="2026-05-05T17:40:12+02:00" level=warning msg="C:\Users\pablo\Documents\DAW\PruebaGlobalModulo\docker-compose.yml: the attribute 'version' is c
it will be ignored, please remove it to avoid potential confusion"
[+] up 12/12
✓ Image httpd:alpine Pulled 7.5s
✓ Network pruebaGlobalModulo_red_daw Created 0.1s
✓ Container daw_app Started 1.1s
PS C:\Users\pablo\Documents\DAW\PruebaGlobalModulo>
```

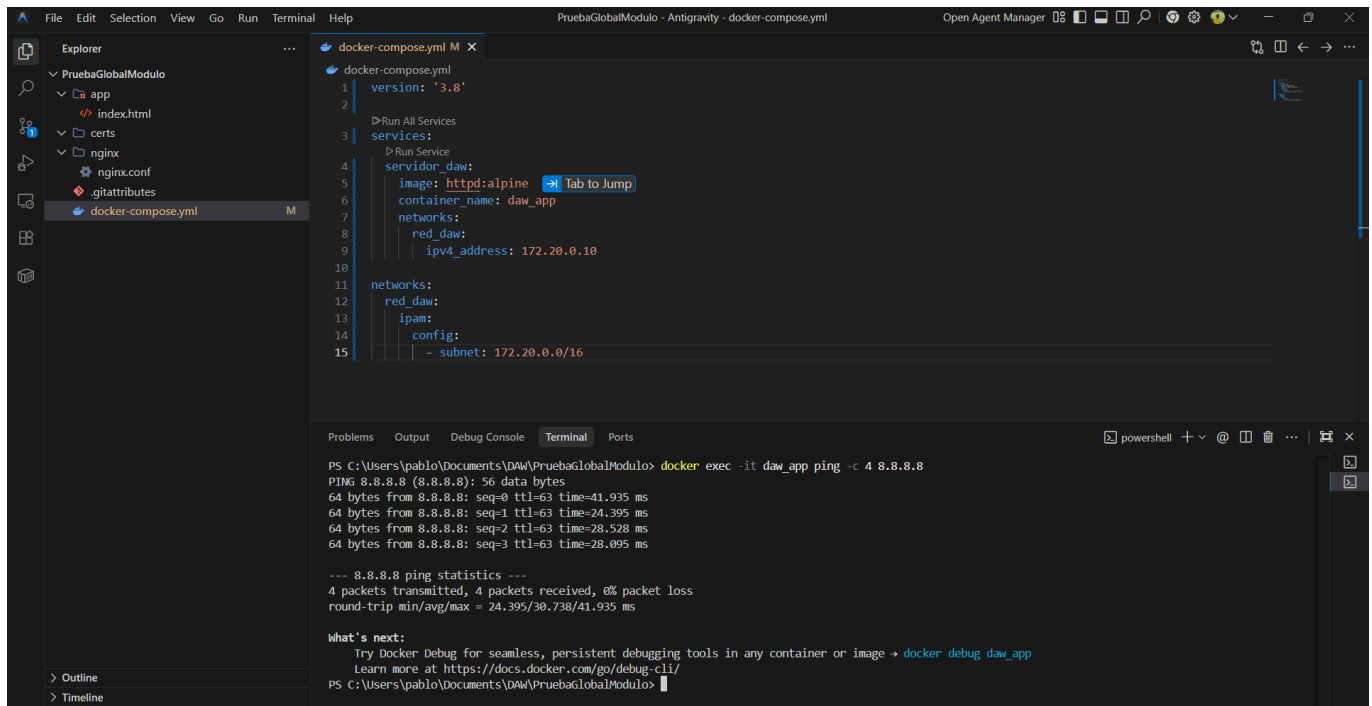
En la captura superior verifico la correcta creación de la red en Docker, confirmando que la subred `172.20.0.0/16` se ha aplicado correctamente en la sección IPAM.

### Comprobación de conectividad básica:

```

PS C:\Users\pablo\Documents\DAW\PruebaGlobalModulo> docker network inspect pruebaglobalmodulo_red_daw
[
  {
    "Name": "pruebaglobalmodulo_red_daw",
    "Id": "31d8c01f6af1e6a09f0834af813a1680dcdf70d11d2123aab3549eb1b22915b",
    "Created": "2026-05-05T15:40:20.417847128Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.20.0.0/16",
          "Gateway": "172.20.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {
      "com.docker.compose.config-hash": "3c247dbd8cb372175b1086776da87bd7a075ee1fdd27a7ec893e001713e52bd9",
      "com.docker.compose.network": "red_daw",
      "com.docker.compose.project": "pruebaglobalmodulo",
      "com.docker.compose.version": "5.1.3"
    },
    "Containers": {
      "36f761dc5997a1947724f9aaf055b2530d837a8ee43a9d6334540e067f8684ab": {
        "Name": "daw_app",
        "EndpointID": "bce19b2f73d6fd907fb0b8ab01deaf0fa0627ab1c2aada1659cadbb532334f8",
        "MacAddress": "be:33:42:a8:84:92",
        "IPv4Address": "172.20.0.10/16",
        "IPv6Address": ""
      }
    }
  },
  {
    "Status": {
      "IPAM": {
        "Subnets": {
          "172.20.0.0/16": {
            "IPsInUse": 4,
            "DynamicIPsAvailable": 65532
          }
        }
      }
    }
  }
]
PS C:\Users\pablo\Documents\DAW\PruebaGlobalModulo>

```



```
version: '3.8'

services:
  servidor_daw:
    image: httpd:alpine
    container_name: daw_app
    networks:
      red_daw:
        ipv4_address: 172.20.0.10

networks:
  red_daw:
    ipam:
      config:
        - subnet: 172.20.0.0/16
```

```
PS C:\Users\pablo\Documents\DAW\PruebaGlobalModulo> docker exec -it daw_app ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=63 time=41.935 ms
64 bytes from 8.8.8.8: seq=1 ttl=63 time=24.395 ms
64 bytes from 8.8.8.8: seq=2 ttl=63 time=28.528 ms
64 bytes from 8.8.8.8: seq=3 ttl=63 time=28.095 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 24.395/30.738/41.935 ms

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug daw_app
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\pablo\Documents\DAW\PruebaGlobalModulo>
```

Mediante el comando `docker exec -it daw_app ping 8.8.8.8` demuestro que el contenedor tiene salida a Internet, enviando y recibiendo paquetes (0% packet loss) de forma exitosa.

## Apartado 2 (RA2): Servidor Web y Proxy Inverso Seguro

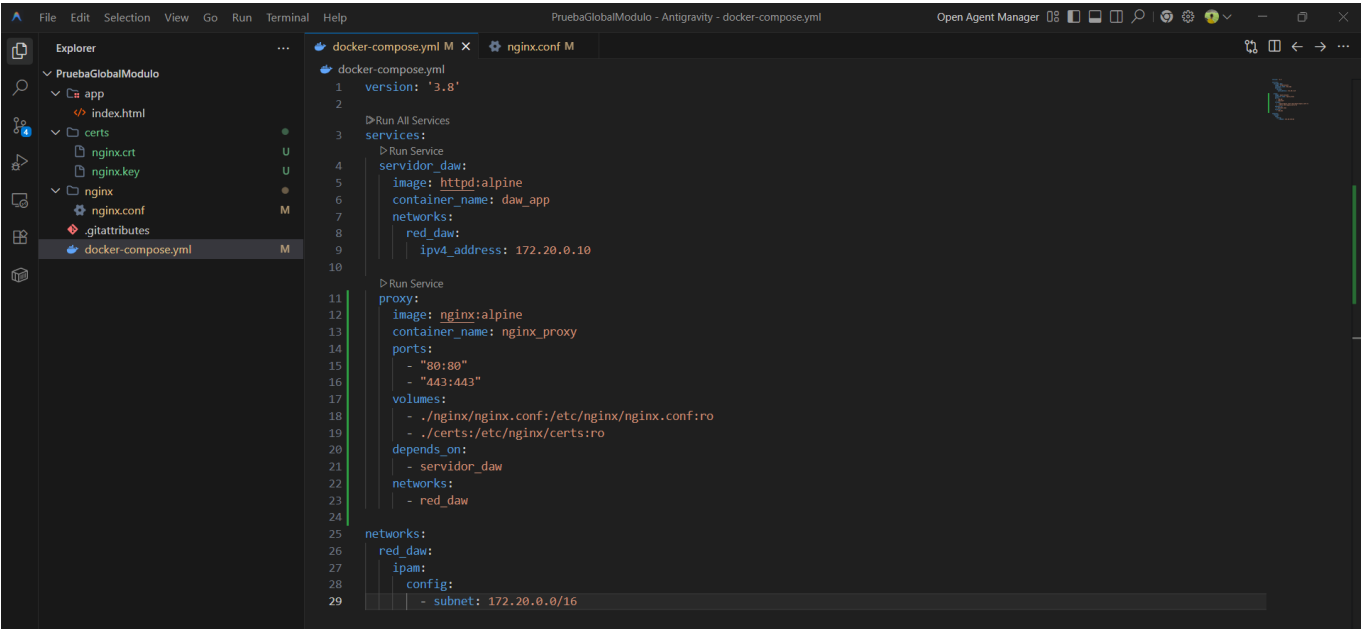
**Objetivo:** Implementar Nginx como proxy inverso y forzar conexiones estrictas por HTTPS utilizando un certificado digital.

Para asegurar las comunicaciones, he añadido un contenedor con Nginx (`nginx:alpine`) que actúa como puerta de entrada a la infraestructura.

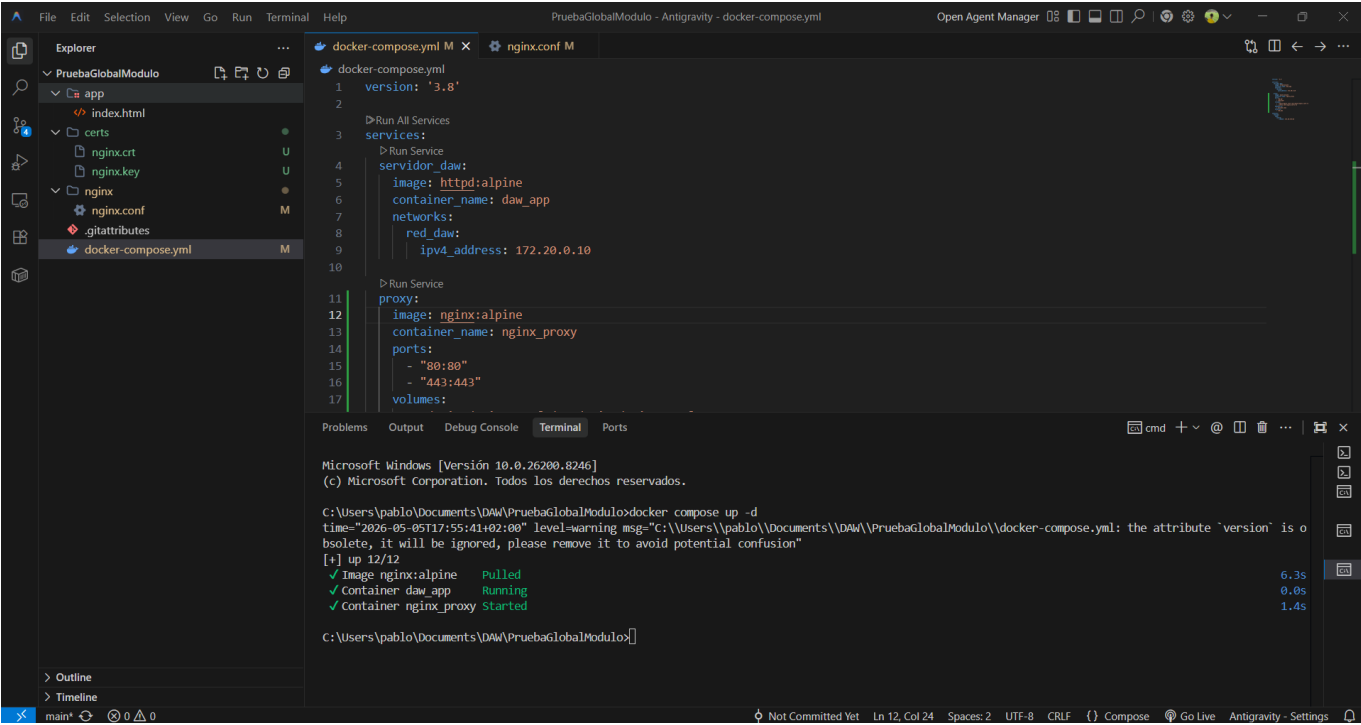
- Generación del Certificado:** He generado un certificado digital autofirmado (`nginx.crt` y `nginx.key`) utilizando la herramienta OpenSSL con una validez de 365 días, asignado al dominio ficticio `prueba-daw.local`.
- Configuración del Proxy (.conf):** En el archivo `nginx.conf`, he configurado un bloque `server` escuchando en el puerto 80 (HTTP) que ejecuta una redirección permanente (`return 301`) hacia el puerto 443 (HTTPS). En el bloque de SSL, utilizo la directiva `proxy_pass` para enrutar el tráfico de forma invisible hacia el puerto 80 interno del contenedor de la aplicación (`daw_app`).

**Capturas de la creacion del certificado:**

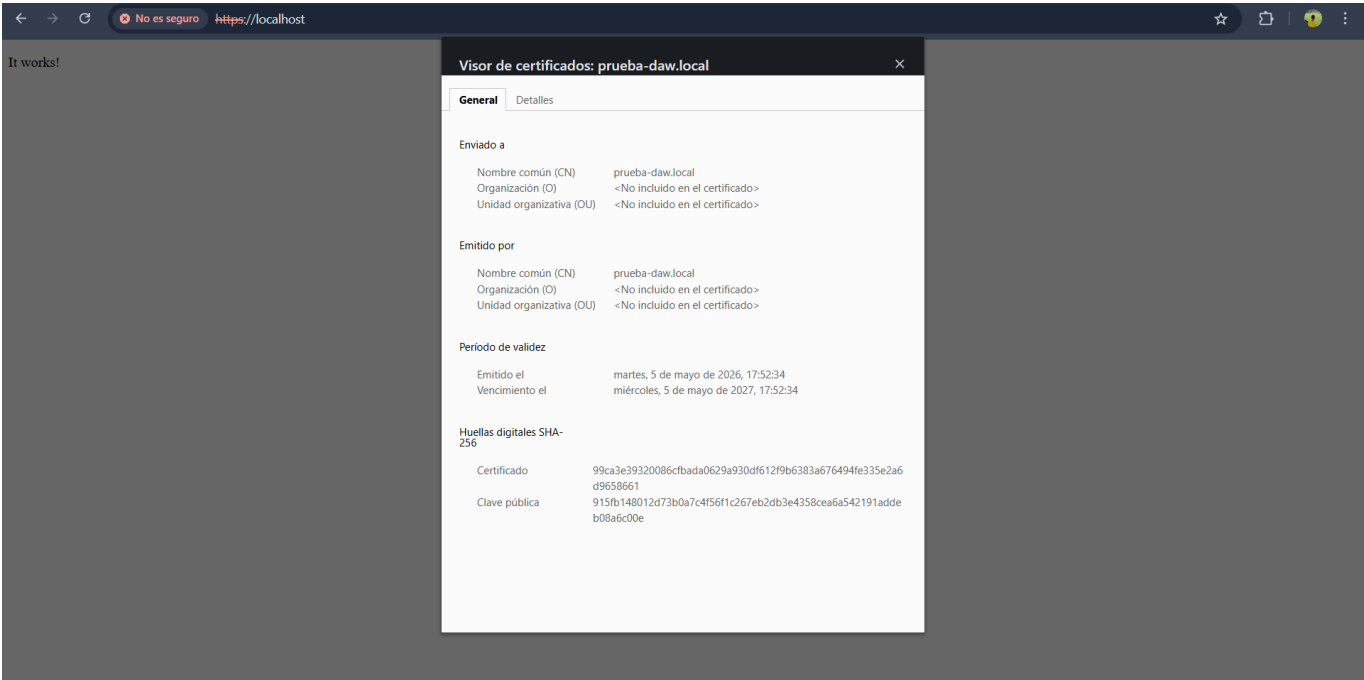




Levantar cambios:



Prueba de funcionamiento y Certificado:



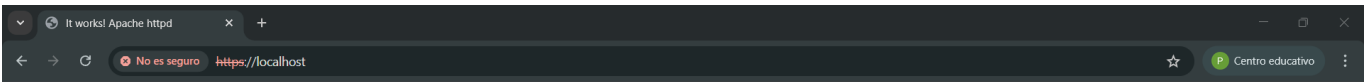
Al acceder al servidor web, Nginx fuerza automáticamente la conexión a HTTPS. El navegador muestra la advertencia esperada al ser un certificado autofirmado, pero la encriptación SSL está activa y encriptando el tráfico.

### Apartado 3 (RA3): Servidor de Aplicaciones y Despliegue

**Objetivo:** Desplegar la aplicación y verificar la comunicación correcta con el proxy inverso.

El código de la aplicación lo he alojado en el directorio local `./app`, y lo he montado como un volumen persistente dentro del contenedor `daw_app` (en la ruta raíz de Apache: `/usr/local/apache2/htdocs/`). Esto permite que el servidor de aplicaciones sirva los archivos sin necesidad de reconstruir la imagen.

**Prueba del funcionamiento:**



Con esta captura evidencio que el despliegue es exitoso. El usuario accede por HTTPS a través del puerto seguro de Nginx, y este se comunica internamente con el contenedor Apache (*daw\_app*) para devolver el contenido, demostrando la correcta integración de ambas capas.

## Apartado 4 (RA4): Servidor de Transferencia de Archivos

**Objetivo:** Instalar un servidor SFTP enjaulado (chroot) para el usuario de despliegue.

He integrado el servicio *atmoz/sftp* en la arquitectura, mapeando el puerto 2222 del host al puerto 22 del contenedor.

- **Configuración del usuario y Chroot:** He aprovisionado el usuario *deploy\_user*. Mediante la configuración de los volúmenes en Docker, he montado el directorio *./app* (donde reside el código web) directamente en */home/deploy\_user/app*. La propia imagen aplica automáticamente una política de "enjaulado" (chroot) sobre el directorio *home* del usuario.

### Añadir el servidor SFTP a la infraestructura:

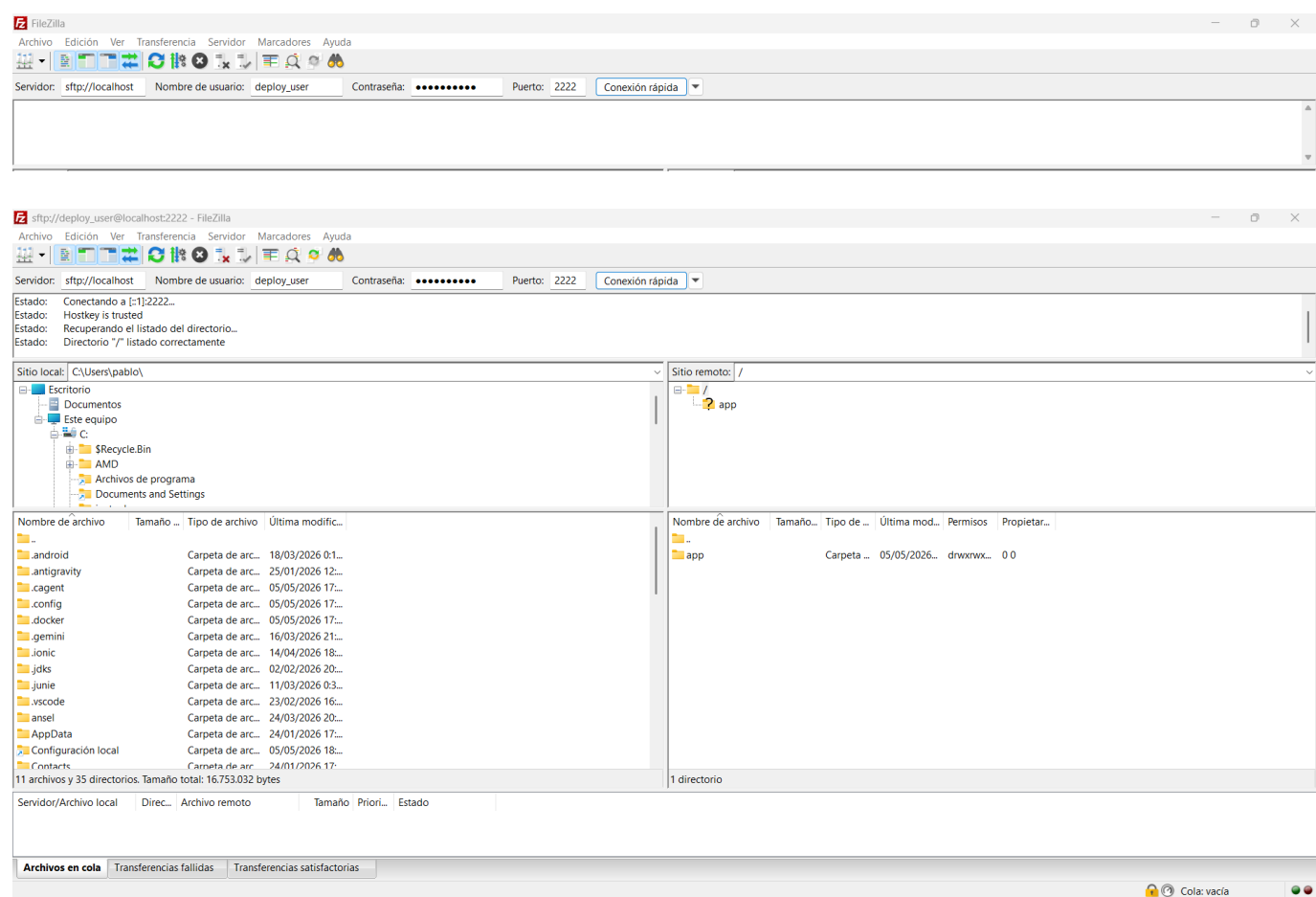
```
1 version: '3.8'
2 services:
3   servidor_daw:
4     image: httpd:alpine
5     container_name: daw_app
6     volumes:
7       - ./app:/usr/local/apache2/htdocs/
8     networks:
9       red_daw:
10        ipv4_address: 172.20.0.10
11
12   proxy:
13     image: nginx:alpine
14     container_name: nginx_proxy
15     ports:
16       - "80:80"
17       - "443:443"
18     volumes:
19       - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
20       - ./certs:/etc/nginx/certs:ro
21     depends_on:
22       - servidor_daw
23     networks:
24       - red_daw
25
26   sftp_server:
27     image: atmoz/sftp:alpine
28     container_name: daw_sftp
29     ports:
30       - "2222:22"
31     command: deploy_user:Secreta123::app
32     volumes:
33       - ./app:/home/deploy_user/app
34     networks:
35       - red_daw
36
37 networks:
38   red_daw:
39     ipam:
40       config:
41         - subnet: 172.20.0.0/16
```

### Levantar SFTP:

```
C:\Users\pablo\Documents\DAW\PruebaGlobalModulo>docker compose up -d
time="2026-05-05T18:09:01+02:00" level=warning msg="C:\\Users\\pablo\\Documents\\DAW\\PruebaGlobalModulo\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 9/9
✓ Image atmoz/sftp:alpine Pulled 3.3s
✓ Container daw_sftp Started 2.5s
✓ Container daw_app Started 2.4s
✓ Container nginx_proxy Running 0.0s

C:\Users\pablo\Documents\DAW\PruebaGlobalModulo>
```

### Prueba de acceso (FileZilla):



\*En el cliente FileZilla confirmo la conexión exitosa al puerto 2222. En la ventana del "Sitio remoto", observo que el directorio raíz / es el límite superior para el usuario. Es imposible navegar a directorios del sistema operativo del servidor, garantizando la seguridad del entorno.

## Apartado 5 (RA5): Resolución de Nombres y Monitorización

**Objetivo:** Configurar el dominio local y monitorizar los registros (logs) en busca de errores provocados.

Para que la infraestructura responda a un dominio personalizado, he editado el archivo de resolución estática del sistema anfitrión (`hosts`), apuntando el dominio ficticio `prueba-daw.local` a la dirección de `loopback` (`127.0.0.1`).

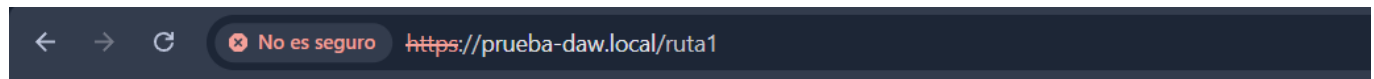
Para probar la monitorización, he lanzado peticiones a recursos inexistentes (`/ruta1`, `/ruta2`) a través del navegador.

**Configurar la resolución de nombres local (El archivo `hosts`)**

```
127.0.0.1 prueba-daw.local
```

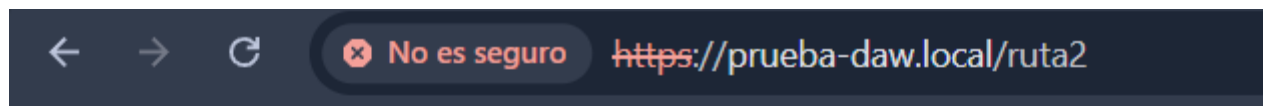
**Provocar tres errores de acceso intencionados**





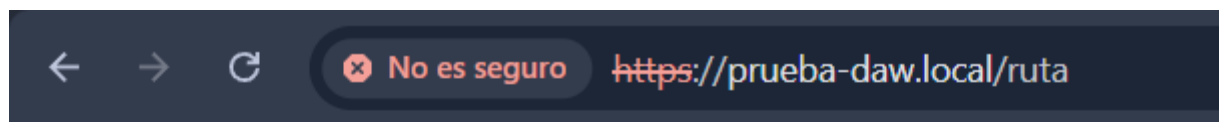
# Not Found

The requested URL was not found on this server.



# Not Found

The requested URL was not found on this server.



# Not Found

The requested URL was not found on this server.

## Extracción del registro de errores:

```
C:\Users\pablo\Documents\DAW\PruebaGlobalModulo>docker logs daw_app
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.20.0.10. Set the 'ServerName' directive globally to suppress this message
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.20.0.10. Set the 'ServerName' directive globally to suppress this message
[Tue May 05 16:09:07.068552 2026] [mpm_event:notice] [pid 1:tid 1] AH00489: Apache/2.4.66 (Unix) configured -- resuming normal operations
[Tue May 05 16:09:07.068702 2026] [core:notice] [pid 1:tid 1] AH00994: Command line: 'httpd -D FOREGROUND'
172.20.0.2 - - [05/May/2026:16:19:33 +0000] "GET /ruta-falsa-1 HTTP/1.1" 404 236
172.20.0.2 - - [05/May/2026:16:19:33 +0000] "GET /favicon.ico HTTP/1.1" 404 236
172.20.0.2 - - [05/May/2026:16:19:44 +0000] "GET /ruta1 HTTP/1.1" 404 236
172.20.0.2 - - [05/May/2026:16:20:06 +0000] "GET /ruta2 HTTP/1.1" 404 236
172.20.0.2 - - [05/May/2026:16:20:38 +0000] "GET /ruta HTTP/1.1" 404 236
C:\Users\pablo\Documents\DAW\PruebaGlobalModulo>
```

## Análisis técnico del registro (Log):

- **172.20.0.2:** Indica la dirección IP de origen de la petición. Al estar detrás de un proxy inverso, Apache no ve la IP real del cliente, sino la IP interna que Docker le ha asignado al contenedor **nginx\_proxy**.
- **GET /ruta-falsa-1:** Muestra el método HTTP utilizado para solicitar el recurso y la ruta exacta que se intentó alcanzar.
- **404:** Es el código de estado de respuesta HTTP (*Not Found*), evidenciando que el servidor procesó la solicitud pero no encontró ningún archivo en dicha ruta, generando el error intencionado.

---

## Apartado 6 (RA6): Control de Versiones y Documentación

**Objetivo:** Documentar el proceso e implementar control de versiones Git.

Durante todo el proceso de despliegue, he gestionado la carpeta de configuración mediante un repositorio Git local. He realizado diversos *commits* lógicos que reflejan la evolución de la infraestructura (creación de la red, proxy inverso, servidor SFTP, etc.).

También he subido el repositorio a GitHub: <https://github.com/pjimpin1207/PruebaGlobalModulo>